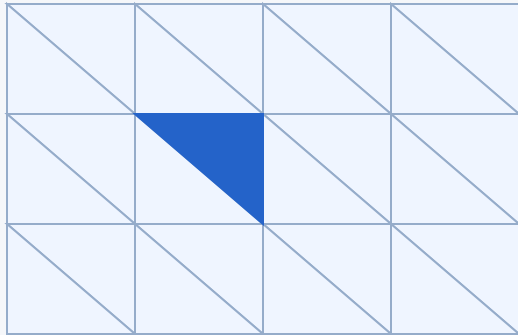


Finite elements → distributed algebra

A brief reminder: local basis functions turn a PDE into a sparse system.

01 Mesh the domain



Small elements, local support

02 Approximate the field

$$u_h = \sum_i u_i \varphi_i$$

DoFs u_i weight basis functions φ_i .

For scalar P1: one DoF per vertex

03 Assemble the system

$$Au = b$$



Sparse coupling between DoFs

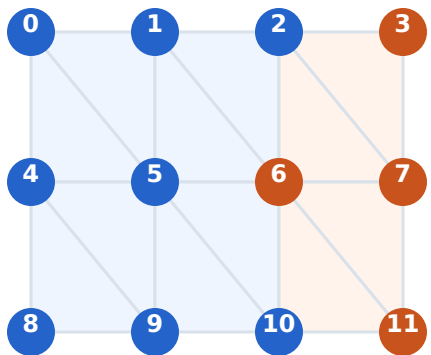
FEniCSx: the platform • DOLFINx: meshes, function spaces and distributed assembly

Local finite-element support creates sparse algebra with distributed ownership.

One mesh, two MPI ranks

Scalar P1 • colors identify the owner • outlines identify local ghost copies

GLOBAL CELL PARTITION

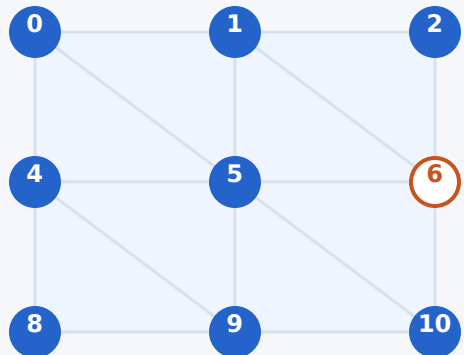


Illustrative ownership;
actual IDs come from IndexMap.

Rank 0

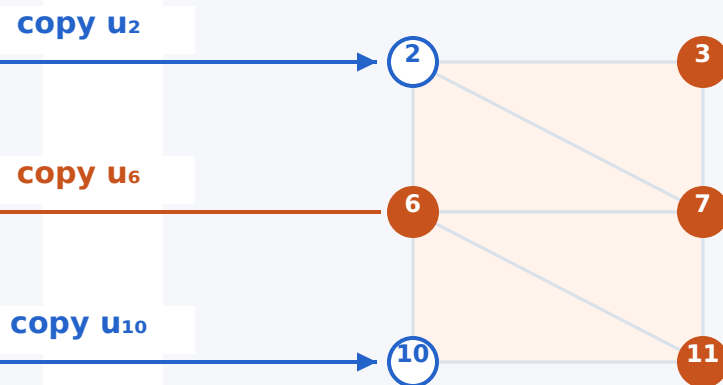
Rank 1

RANK 0 / 8 owned + 1 ghost



[owned values | ghost copies]

RANK 1 / 4 owned + 2 ghosts



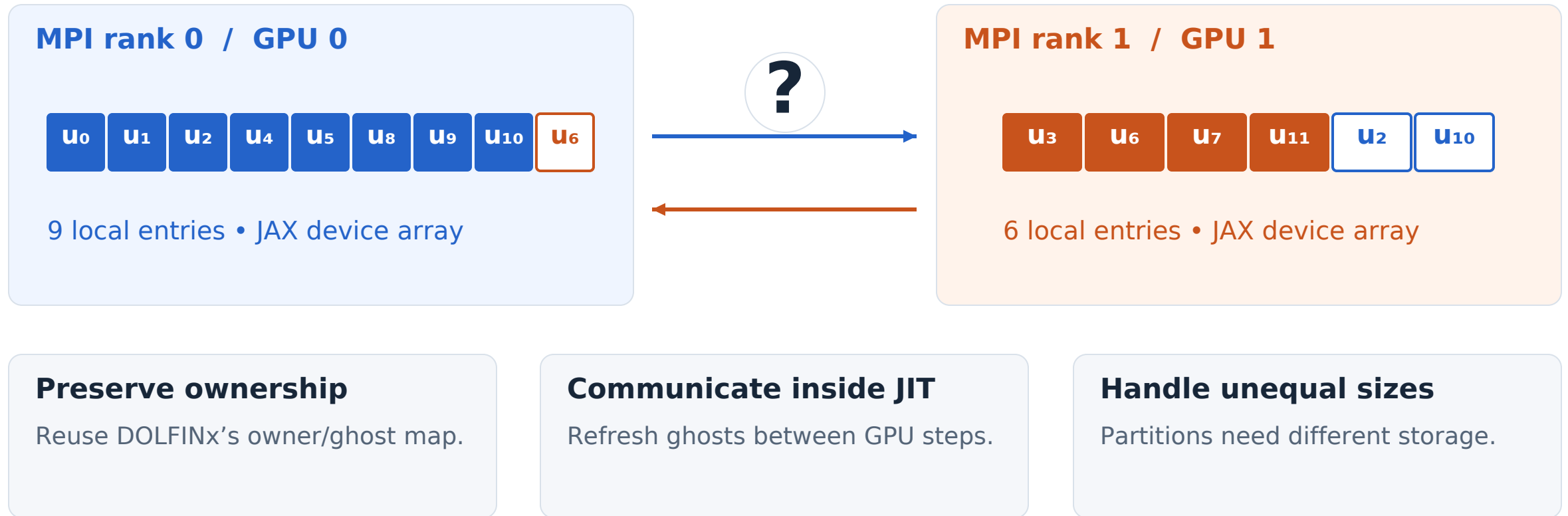
[owned values | ghost copies]

Each DoF has one owner. Other ranks keep copies where local work needs them.

Ghost DoFs are copied coefficients; ghost cells are additional mesh cells (not shown here).

How do we preserve this on GPUs with JAX?

Keep the FEM ownership model. Move repeated numerical work onto the devices.



How do remote owner values reach the right local ghost slots?

A local matrix row needs a remote vector value

A four-DoF algebraic toy • row partitioning • zero-based indices • start with $y = 0$

1 SPLIT THE MATRIX BY OWNED ROWS

$$\begin{array}{c}
 \mathbf{A} \\
 \begin{array}{c} 0 \\ 1 \\ 2 \\ 3 \end{array}
 \begin{bmatrix}
 2 & -1 & 0 & 0 \\
 -1 & 2 & -1 & 0 \\
 0 & -1 & 2 & -1 \\
 0 & 0 & -1 & 2
 \end{bmatrix}
 \end{array}
 \times
 \begin{array}{c}
 \mathbf{x} \\
 \begin{array}{c} 1 \\ 2 \\ 4 \\ 8 \end{array}
 \end{array}
 =
 \begin{array}{c}
 \mathbf{Ax} \\
 \begin{array}{c} 0 \\ -1 \\ -2 \\ 12 \end{array}
 \end{array}$$

The matrix stays local; selected x values travel.

2 EXCHANGE x , THEN MULTIPLY LOCAL ROWS

Rank 0 needs remote x_2

1 2 4

$$y_1 = -1 \cdot 1 + 2 \cdot 2 - 1 \cdot 4 = -1$$

Rank 1 needs remote x_1

4 8 2

$$y_2 = -1 \cdot 2 + 2 \cdot 4 - 1 \cdot 8 = -2$$

A remote column reference becomes a local ghost-vector lookup.

Our API computes owned $y += Ax$; x and matrix inputs are preserved, and output ghosts stay unchanged.

mpi4jax: pack → sendrecv → unpack

The ownership plan is prepared once; the JIT-compiled operation exchanges current values.



GPU buffers can follow different transport paths

HOST-STAGED

GPU → CPU → MPI → CPU → GPU

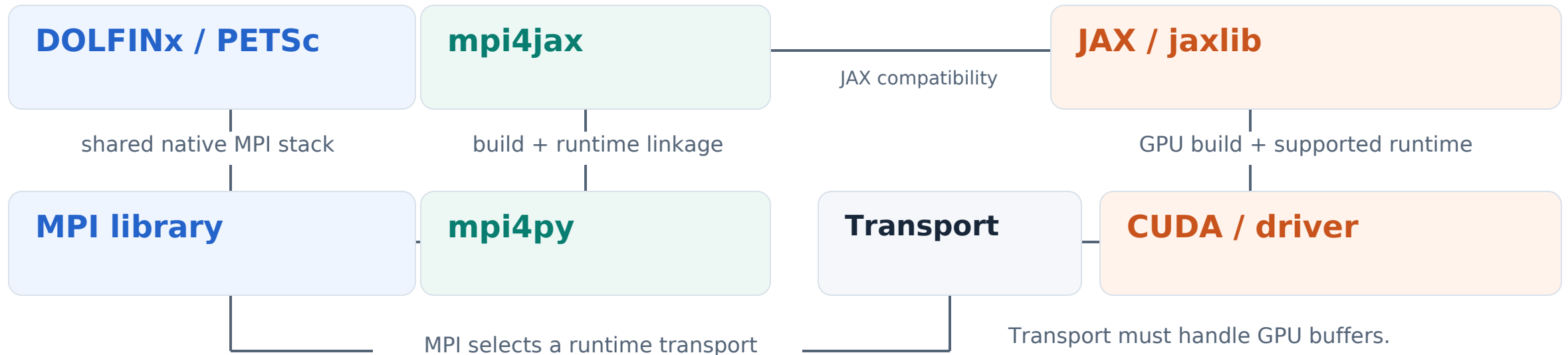
CUDA-AWARE MPI

GPU pointer → MPI → GPU pointer

CUDA-aware MPI accepts device buffers; the actual transport may still stage on the host.

The challenge: make the whole HPC stack agree

Python packages, native libraries and the selected runtime transport must be compatible.



01 CUDA discovery across Spack + venv

mpi4py and CUDA lived in different prefixes.
The mpi4jax build needed a discovery fix.

02 Selected UCX lacked GPU support

The loaded transport misread a device buffer.
A working MPI import did not validate this path.

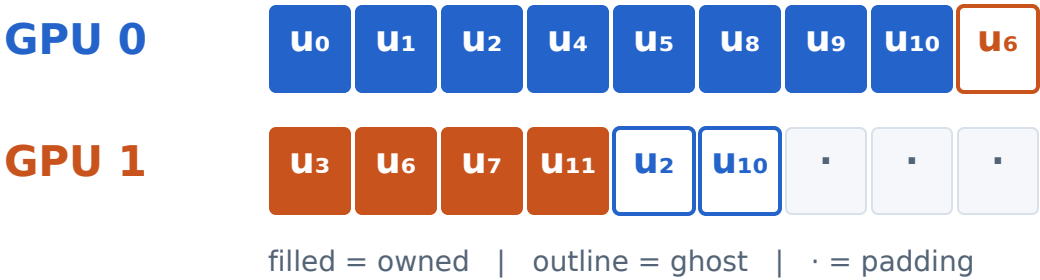
Successful installation is only the start: validate the GPU exchange in the actual job.

Explicit ghost exchange with JAX sharding

One rank per GPU • retain DOLFINx ownership • let JAX execute the numerical collective

ONCE / HOST DOLFINx IndexMap + mpi4py metadata → packing indices, receive slots, masks
Initialize distributed JAX; create a one-dimensional device mesh with axis “rank”.

GLOBAL ARRAY SHAPE [2, 9] / ONE ROW PER GPU



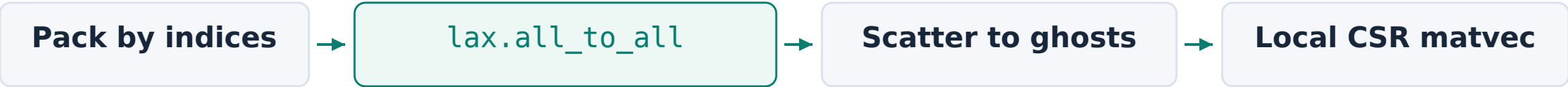
Explicit packets preserve the ghost map

GPU 0 sends [u₂, u₁₀] → GPU 1 ghost slots

GPU 1 sends [u₆, 0*] → GPU 0 ghost slot

* Fixed-width packet padding is masked / discarded.

EACH JIT-COMPILED CALL / `shard_map` runs the same body on every device



Sharding places the arrays. Our plan defines which owner values fill which ghosts.
Current scope: forward INSERT + scalar CSR matvec. Tradeoff: padding and a global all-to-all collective.

Newer JAX APIs: optimization opportunities

Candidates for our sharded implementation • preserve the ownership map • benchmark on the target GPUs

Less packet padding

`lax.ragged_all_to_all`

Fixed peer packets



Packed values + counts / offsets



Send variable-length slices.
Keep fixed buffer capacities;
vector / CSR padding remains.

Reuse device storage

`donate_argnums` / `jax.new_ref`



Reuse buffers or write ghost slots.

Donation consumes the input.
Refs enable explicit mutation.
Measure memory + dispatch cost.

Overlap useful work

`lax.psend` / `lax.precv`

Target schedule — verify in a trace

communicate ghost values

owned-column part

ghosts

Split owned / ghost-column work.
Test scheduling and GPU progress.
MPI-style overlap is not automatic.

Also profile local CSR alternatives; consider Pallas fusion only for a measured kernel bottleneck.

First test storage reuse and ragged exchange; measure end-to-end matvec before choosing.

Optimization candidates, not measured speedups. Verify GPU execution, numerical results and required JAX transforms.